

Име: _____, ФН: _____, Спец.: _____

Задача	1	2	3	Общо
получени точки				
максимум точки	10	10	10	30

Задача 1 (10 точки)

Нека имаме предположения за цените на една акция през следващите n дни, записани в редицата `price[1..n]`. Избираме два дни $1 \leq i < j \leq n$, такива че на ден i купуваме акцията за цена `price[i]`, а на ден j продаваме акцията за цена j .

Предложете алгоритъм със сложност $O(n)$, който решава проблема MAXPROFIT:

Вход: `P[1..n]` – масив от естествени числа, описващи цената на акцията за следващите n дни

Изход: `profit` – максималната печалба, която можем да направим, ако извършим най-много една покупко-продажба.

Задача 2 (10 точки)

Предложете алгоритъм със сложност $O(n \log n)$, който решава проблема 2SUM:

Вход: `A[1..n]`, s – масив от естествени числа и естествено число

Изход: `true`, ако съществуват индекси $1 \leq i < j \leq n$, такива че $A[i] + A[j] = s$.
`false` в противен случай.

Задача 3 (10 точки)

Даден е списък от m низа от цифри $S = \{s_1, s_2, \dots, s_m\}$. Ще означаваме с $n = \sum_i |S[i]|$ Предложете алгоритъм със сложност $O(n \log n)$, който решава проблема MAXCONCAT:

Вход: `S[1..m]` – масив от низове, съставени от цифри;

Изход: `num` – максималното число, което може да бъде получено, конкатенирайки числата едно след друго в някаква пермутация на масива S .

Решение на зад. 1:

Формално отговорът на задачата е $\max_{1 \leq i < j \leq n} \{P[j] - P[i]\} = \max_{1 \leq i \leq n} \{P[i] - \min_{1 \leq i < j} P[j]\}$. Ще обходим линейно масива P , поддържайки две променливи:

1. **answer**, в която записваме максималната печалба до момента;
2. **buy**, в която записваме най-добрия предходен ден за покупка на акцията (този, в който акцията е била най-евтина).

Algorithm 1: MaxProfit

Input: $P[1..n]$ – array of natural numbers

```
1 answer  $\leftarrow 0$ 
2 buy  $\leftarrow \infty$ 
3 for  $i \leftarrow 1$  to  $n$  do
4   answer  $\leftarrow \max\{\text{answer}, P[i] - \text{buy}\}$ 
5   buy  $\leftarrow \min\{\text{buy}, P[i]\}$ 
6 return answer
```

На ред 2, стойността на отговора почва от 0, защото може да не направим покупко-продажба и да нямаме печалба. На ред 3 стойността на **buy** е ∞ , защото предходен на първия ден няма.

Редове 4 и 5 се грижат за поддържане на променливите, както сме ги описали горе.

Сложността очевидно е $O(n)$, защото се изпълнява единствено цикъла на ред 3, а той има по една итерация за всяко $i = 1, 2, \dots, n$.

Решение на зад. 2:

Ще сортираме масива $A[1..n]$ във време $O(n \log n)$ чрез MERGESORT примерно. От тук нататък с A ще означаваме сортираната версия на масива.

Ще започнем да търсим желаните индекси чрез две показалки, започващи от позиции 1 и n – ще ги наричаме лява и дясна показалка съответно.

Ще разгледаме три случая относно стойността на $A[l] + A[r]$:

1. Ако $A[l] + A[r] = s$, тогава сме намерили търсената двойка индекси и можем да терминираме алгоритъма и да върнем **true**;
2. Ако $A[l] + A[r] > s$, тогава за всяко $l < i < r$ ще е вярно, че $A[i] + A[r] > A[l] + A[r] > s$ и би било безсмислено да търсим двойката със същото r , но по-голямо l : ще намалим r с единица;
3. Ако $A[l] + A[r] < s$, тогава за всяко $l < i < r$ ще е вярно, че $A[l] + A[i] < A[l] + A[r] < s$ и аналогично на горния случай разсъждаваме, че ще увеличим l с единица.

Финално ще отбележим, че ако $l \geq r$, можем да заключим, че двойка индекси, отговаряща на условието няма и да сигнализираме това с **false**.

Algorithm 2: 2Sum

Input: $A[1..n]$, s – array of natural numbers and a natural number

```
1 MERGESORT( $A[1..n]$ )
2  $l \leftarrow 1$ 
3  $r \leftarrow n$ 
4 while  $l \leq r$  do
5   if  $A[l] + A[r] = s$  then
6     return true
7   if  $A[l] + A[r] > s$  then
8      $r \leftarrow r - 1$ 
9   if  $A[l] + A[r] < s$  then
10     $l \leftarrow l + 1$ 
11 return false
```

На ред 1 сортираме A във възходящ ред във време $O(n \log n)$. На редове 2 и 3 създаваме указаните показалки. Цикълът на ред 4 проверява трите условия както сме ги описали по-горе.

Ще отбележим, че на всяка стъпка на while-цикъла на ред 4 количеството $r - l$ намаля с единица (освен ако алгоритъмът на завършва). Понеже $r - l = n - 1$, ще се изпълнят най-много $O(n)$ итерации. Сумарно сложността е $O(n) + O(n \log n) = O(n \log n)$.

Решение на зад. 3:

Дефинираме бинарна релация $\prec$ върху множеството от низове S . За всеки два низа $x, y \in S$, казваме че $x \prec y$ тогава и само тогава, когато конкатенацията $y \cdot x$ е лексикографски по-голяма от $x \cdot y$ (където $\cdot$ е оператор за конкатенация). Алгоритъмът се състои в сортиране на низовете съгласно тази релация.

Да допуснем, че съществува оптимална подредба P , в която два съседни низа A и B са в ред (A, B) , но $B \cdot A > A \cdot B$. Ако разменим местата им, получавайки подредба P' , единствената промяна в конкатенираното число ще бъде замяната на поднизата $A \cdot B$ с $B \cdot A$. Тъй като $B \cdot A > A \cdot B$, то P' ще представлява по-голямо число от P , което противоречи на допускането за оптималност на P . Следователно в оптималната наредба за всеки два съседни елемента е изпълнено $x \cdot y \geq y \cdot x$.

Algorithm 3: MaxConcat

Input: $S[1..m]$ – array of strings

```
1 MERGESORTCUSTOM( $S, 1, m$ )
2 answer  $\leftarrow \varepsilon$  for  $i \leftarrow 1$  to  $m$  do
3   answer  $\leftarrow$  answer  $\circ S[i]$ 
4 return answer
```

За да бъде релацията валидна за сортиране, тя трябва да удовлетворява свойствата за линейна наредба:

1. Ирефлексивност: $x \cdot x > x \cdot x$ е винаги лъжа.
2. Асиметричност: Ако $x \cdot y > y \cdot x$, то $y \cdot x > x \cdot y$ е лъжа.
3. Транзитивност: Нека x, y, z са низове. Нека $v(x)$ е числовата стойност на низа, а $L(x) = 10^{|x|}$. Конкатенацията $x \cdot y$ е $v(x) \cdot L(y) + v(y)$. Условието $x \cdot y > y \cdot x$ е еквивалентно на:

$$v(x)L(y) + v(y) > v(y)L(x) + v(x) \iff v(x)(L(y) - 1) > v(y)(L(x) - 1) \iff \frac{v(x)}{L(x) - 1} > \frac{v(y)}{L(y) - 1}$$

Тъй като релацията $>$ над реалните числа е транзитивна, то ако $\frac{v(x)}{L(x) - 1} > \frac{v(y)}{L(y) - 1}$ и $\frac{v(y)}{L(y) - 1} > \frac{v(z)}{L(z) - 1}$, следва $\frac{v(x)}{L(x) - 1} > \frac{v(z)}{L(z) - 1}$, което доказва транзитивността.

Единственото съществено от към времева сложност тук е сортирането, което работи за $O(\sum_i |S[i]| \log \sum_i |S[i]|)$.